Security Audit — orchard-webapp (pre-rollout)

> Read-only audit performed 2026-09-07 across three tracks: (1) cookie/session/token storage and sensitive-data logging, (2) the two Nitro server routes and secrets/config management, (3) XSS surfaces and authorization gates on money-moving actions. No code was changed as part of this audit — findings only, with a proposed remediation plan to revisit.

What's already clean (confirmed, no action needed)

- No `console.log` / `console.error` /etc. anywhere leaking sensitive data (tokens, passwords, PINs, BVN/NIN).
- No `localStorage` usage anywhere in the app.
- No secrets ever committed to git history (checked full history, not just current state).
- `.gitignore` correctly excludes real-PII local files (`ORCHARD_HANDOFF.md`, the Postman collection, `resolve_bn.md`, `.env`).
- `pnpm audit` shows no known-vulnerable dependencies.
- `PinInput.vue` already sets `autocomplete="off"` on every digit box app-wide.
- Password/PIN change already correctly send `current_password` / `current_pin` to the backend (not just collected and dropped).
- `useFlowState` / `idle-logout` (sessionStorage-backed, in-memory-only) are correctly scoped with no leakage.
- `auth.global.ts`'s route gating (`PUBLIC_ROUTES`) is accurate and runs on every navigation, client and server.

Findings

CRITICAL

1. Unsanitized `v-html` rendering of backend HTML, with no sanitizer library in the app at all.

`product.description` / `product.investment_note` (`app/pages/investments/products/[slug].vue:142,145`) and `type.description` (`app/pages/opportunities/index.vue:45`, `app/pages/investments/types/[id].vue:41`) are injected as raw HTML with zero sanitization anywhere in the pipeline. No `dompurify` / `sanitize-html` /equivalent exists in `package.json`. Whether or not this content is "admin-only" today, this repo has no visibility into that admin panel's own security model — a single compromised admin account, or a bug on the CMS/authoring side, becomes stored XSS executing in every investor's authenticated session. This is elevated from "hygiene issue" to critical by finding 2.

2. Auth cookies have no `secure` / `sameSite`, and the entire `UserProfile` — including the bcrypt transaction-PIN hash, BVN, NIN, and bank account number — is persisted into a plain, JS-readable cookie.

`app/plugins/auth.ts:13-14,28`. `httpOnly` genuinely can't be added without a bigger architecture change (the token must be JS-readable to build the `Authorization` header under this bearer-token SPA model) — accepted as a known trade-off. But `secure` / `sameSite` are trivial to add, and persisting the *entire* user object (rather than just the token, re-fetching the user afterward) is unnecessary: it hands anything that can read cookies a bcrypt hash of

a 6-digit PIN (a tiny, brute-forceable keyspace) plus BVN/NIN/bank details, none of which need to live in cookie storage. Combined with finding 1, this is a complete session + PII exfiltration chain from a single XSS.

3. The Paystack account-resolve endpoint has zero authentication and no rate limiting.

`server/api/paystack/resolve-account.get.ts`. It's a plain Nitro route, not covered by `auth.global.ts` (page-navigation middleware has no effect on direct API calls). Anyone on the internet can call it directly to resolve any Nigerian bank account number to a real account-holder name — for free, off Orchard's Paystack quota, with no limit. This is independently a PII-disclosure/fraud-enablement risk (name-from-account-number resolution is a known social-engineering building block) on top of the quota-abuse cost.

HIGH

4. The invest flow has no PIN/2FA re-confirmation before placing an order.

`app/pages/investments/products/[slug].vue` — only a terms-checkbox gates `POST /users/investments/order/invest`. `SendMoneyModal.vue` already has a working amount→PIN two-step pattern for withdrawals; Invest, the highest-value money-moving action in the app, has no equivalent step-up check at all.

MEDIUM

5. No HTTP security headers anywhere.

No CSP, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, or HSTS — no module or manual config sets any of these. A CSP in particular would meaningfully contain the blast radius of finding 1 even if a sanitization gap ever slips through later.

6. Missing/inconsistent `autocomplete` attributes.

BVN/NIN inputs in `KycModal.vue` have none (should be `off`); the three password inputs in `SecuritySettingsModal.vue` have none (should be `current-password` / `new-password` / `new-password`, matching the pattern already used correctly on `login.vue` / `signup.vue` / `reset-password.vue`).

7. Paystack publishable key hardcoded as a literal.

`paystackPublicKey` is a hardcoded literal in committed `nuxt.config.ts` with no env-var indirection (unlike `paystackSecretKey`) — switching test/live Paystack mode requires a code change + redeploy rather than an environment change, a real rollout-process risk (shipping the wrong mode unnoticed).

8. Disabling 2FA has no re-confirmation of any kind.

`SecuritySettingsModal.vue`'s `toggleTwoFactor` — a single click on the toggle weakens account security with no step-up check, unlike password/PIN changes which correctly require the current credential.

NEEDS BACKEND CONFIRMATION — CANNOT BE FIXED FROM THIS REPO ALONE

- Whether `GET /users/investments/portfolio/items/:uuid` (and similarly the product/slug endpoint) actually scopes results to the authenticated user server-side, or would return any record for any valid UUID regardless of owner (a potential IDOR) — the frontend has no way to verify this from here.
- Whether wallet-funding bank-transfer reconciliation genuinely happens via backend webhook/reconciliation independent of the "I have sent the money" button (it should — that button fires no backend call at all today,

purely client-side polling) — worth an explicit confirmation that crediting doesn't depend on a user clicking anything.

- Whether `ship.crowdital.com` runs with verbose/debug error output in production — the generic proxy (`server/api/proxy/[...path].ts`) relays whatever the backend returns straight to the client unfiltered, so any backend debug leakage would pass through un-sanitized.

Proposed remediation plan (not yet implemented — for review)

Files: `app/plugins/auth.ts`, `app/stores/auth.ts`, new `app/utils/sanitizeHtml.ts` (or inline per page), `package.json`, `app/pages/investments/products/[slug].vue`, `app/pages/opportunities/index.vue`, `app/pages/investments/types/[id].vue`, `server/api/paystack/resolve-account.get.ts`, `nuxt.config.ts`, `app/components/kyc/KycModal.vue`, `app/components/settings/SecuritySettingsModal.vue`

- **Sanitize all four `v-html` sinks.** Add `isomorphic-dompurify` (works in both SSR and client render — needed since `v-html` content renders during SSR too, not just client-side) and run `product.description` / `product.investment_note` / `type.description` through it with a strict tag/attribute allow-list (`p, br, strong, em, ul, ol, li, a[href], h1-h4, blockquote`; `a[href]` restricted to `http(s)`, `rel="noopener noreferrer"` enforced) before rendering.
- **Cookie hardening** (`app/plugins/auth.ts`): add `secure: true, sameSite: 'lax'` to both cookies; shorten `COOKIE_MAX_AGE` to 7 days (agreed with the product owner — balances convenience vs. exposure window); stop persisting the full `UserProfile` into `oc_auth_user` — persist the token only, and have the plugin re-fetch `/users/dashboard` to repopulate `authStore.user` on rehydration instead of reading it back from a cookie. Drop the now-unnecessary `oc_auth_user` cookie entirely.
- **Lock down the Paystack resolve-account route** (`server/api/paystack/resolve-account.get.ts`): require an `Authorization` header, and verify it by calling the real backend (e.g. `GET https://ship.crowdital.com/api/users/dashboard` with that header) before performing the Paystack lookup — reject with 401 if that check fails. Add a simple in-memory per-IP rate limiter (good enough for launch; flagged as needing a shared store like Redis if this ever runs across multiple server instances).
- **Add a PIN confirmation step to the invest flow**, mirroring `SendMoneyModal.vue`'s `step: 'amount' | 'pin'` pattern, before `POST /users/investments/order/invest` fires.
- **Add baseline security headers** via Nitro `routeRules` in `nuxt.config.ts` (no new dependency needed for a handful of static headers) — CSP (allowing the existing Paystack script origin), `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin`, and HSTS.
- **Fix `autocomplete` attributes**: `off` on `KycModal.vue`'s account number/BVN/NIN fields; `current-password` / `new-password` / `new-password` on `SecuritySettingsModal.vue`'s three password fields.
- **Make `paystackPublicKey` env-driven** in `nuxt.config.ts`, keeping the current test key as the local-dev default/fallback.
- **Add a current-PIN (or password) re-confirmation before disabling 2FA** in `SecuritySettingsModal.vue`, consistent with how password/PIN changes already require the current credential.

Verification (once implemented)

- `npx vue-tsc --noEmit -p .nuxt/tsconfig.json`.

- `npm run dev`: confirm a product/type description with embedded `<script>` / `onerror` markup renders inertly (no execution) after sanitization; confirm normal formatting (bold/lists/links) still displays correctly.
- Inspect cookies in dev tools after logging in: confirm `Secure` / `SameSite=Lax` flags are present, `oc_auth_user` no longer exists, and `oc_auth_token` expires in ~7 days.
- Hit `/api/paystack/resolve-account` directly with curl (no Authorization header) and confirm it now 401s instead of resolving.
- Walk through the invest flow and confirm a PIN step is now required before an order is placed; confirm Send Money's existing PIN step is unaffected.
- Check response headers (`curl -I`) for the new security headers.
- Confirm KYC/BVN/NIN fields and Settings password fields no longer offer inappropriate browser autofill suggestions.